

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: REYES MUÑOZ FRANCISCO XAVIER

PARALELO: A

SEMESTRE: QUINTO SEMESTRE

ACTIVIDAD: Actividad#1: Estudio de Caso – Sistemas de Gestión de Incidentes (Help Desk)

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividad # 1

Sistema de Gestión de Incidentes Técnicos Help Desk para Infraestructura de Servidores

Análisis del problema

Identificar los actores principales (Usuario final, Técnico de soporte, Administrador de sistemas).

La gestión manual o desorganizada de los incidentes técnicos dentro de un Data Center puede ocasionar tardanzas en su atención, duplicidad de registros, No se tiene control del estado real de cada incidente en la operación, por la asignación de técnicos inadecuados.

El sistema que se propone resolver estos problemas con una plataforma lógica Help Desk que centralice la información de los incidentes, Vamos a comenzar por definir cuáles serán las funciones más relevantes.

Actores principales.

Actor	Descripción	Funciones principales
Usuario final	Persona que reporta un fallo técnico relacionado con servicios, servidores, red o aplicaciones.	• Crear tickets • Consultar estado del incidente • Recibir notificaciones
Técnico de soporte	Profesional encargado de atender y resolver incidentes asignados.	• Revisar tickets • Actualizar estados • Documentar solución



		• Cerrar incidentes
Administrador de sistemas	Responsable de gestionar usuarios, técnicos, prioridades, categorías y supervisar el flujo de incidentes.	• Asignar técnicos • Escalar incidentes • Monitorear métricas • Administrar el sistema

Requerimientos funcionales

Código	Requerimiento funcional
RF01	El sistema debe permitir crear tickets de incidentes técnicos.
RF02	El sistema debe permitir clasificar los incidentes por tipo: hardware o software.
RF03	Los técnicos de soporte deben ser asignados en función de sus habilidades específicas.
RF04	El sistema debe permitir cambiar el estado del ticket: abierto, asignado, en proceso, resuelto y cerrado.
RF05	El sistema debe enviar notificaciones automáticas a los técnicos cuando se asignen nuevos incidentes.
RF06	El sistema debe proporcionar al administrador la capacidad de escalar incidentes.
RF07	El sistema necesita mantener un registro de las modificaciones realizadas para auditoría y seguimiento.
RF08	Debe ser posible consultar el estado de los incidentes reportados en el sistema.
RF09	El sistema debe permitir generar reportes básicos sobre incidentes atendidos, pendientes y críticos.
RF10	El sistema deberá admitir la autenticación de usuarios por rol.

Requerimientos no funcionales



Código	Requerimiento no funcional
RNF01	El sistema debe ser escalable para incorporar nuevas categorías de incidentes.
RNF02	Para mantener la sostenibilidad del sistema, la lógica empresarial debe estar separada de los datos y las interfaces.
RNF03	El sistema debe lograr la rastreabilidad de las modificaciones del historial mediante control de versiones en GitHub.
RNF04	El sistema debería permitir la reutilización de componentes mediante patrones de diseño.
RNF05	El sistema debe regular los roles de usuario para una implementación de seguridad sólida en las funciones administrativas.
RNF06	El sistema debe estar estructurado con una arquitectura coherente y documentada.
RNF07	El sistema necesita permitir futuras integraciones con correo electrónico y sistemas de monitoreo.

Propuesta de arquitectura lógica

El esquema recomendado se basa en el esquema MVC (Model View Controller) y se divide en tres etapas del sistema

Capa	Función
Modelo	Representa las entidades del negocio: Ticket, Usuario, Técnico, Incidente y Notificación.
Vista	Es la representación gráfica con la que interactúa el usuario.
Controlador	El esquema recomendado se basa en el esquema MVC (Model View Controller) y se divide en tres etapas del sistema.

Asimismo, se incorporan los siguientes patrones:

Patrón	Uso dentro del sistema
Singleton	Permite gestionar una sola instancia del gestor central de tickets.
Factory Method	Permite generar incidentes de hardware, red o software.



Observer	Habilita enviar informes automáticos a técnicos y administradores De que se trata el cambio.
MVC	Se utiliza para la estructuración y organización del sistema.

Gitflow propuesto para el repositorio

➤ Justificación del uso de Gitflow

Gitflow es un modelo de trabajo con ramas que permite organizar el desarrollo de software de forma controlada. Su uso resulta adecuado para este proyecto porque permite separar la versión estable del sistema, las funcionalidades en desarrollo y las correcciones futuras.

Gitflow, que permite la trazabilidad de los cambios, en este sistema Help Desk, opta por ediciones que no afecten la rama principal ni la operación esencial, permitiéndonos seguir el avance de nuestro proyecto mediante ramas.

➤ Estructura de ramas

Rama	Propósito
master	Contiene la versión estable y final del sistema.
develop	Contiene la integración de funcionalidades en desarrollo.
feature/crear-ticket	Desarrollo de la funcionalidad para crear tickets.
feature/asignar-tecnico	Desarrollo de la funcionalidad para asignar técnicos.
feature/notificaciones	Desarrollo del módulo de notificaciones.
feature/patrones-diseno	Implementación de patrones de diseño lógica.
release/v1.0	Preparar la primera versión estable.



Evidencias

Captura del repositorio público creado.

Popular repositories

[HelpDesk-Tarea](#) Public

Actividad #1: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

3 contributions in the last year

Contribution settings

2026

2025

Francisco Reyes
freyes7194-svg

Edit profile

freyes7194@utm.edu.ec

Highlights

PRO

Contribution activity

May 2026

Created 1 commit in 1 repository

[freyes7194-svg/HelpDesk-Tarea](#) 1 commit

```
Git CMD
C:\Users\franc>cd ..
C:\Users>cd ..
C:\>cd proyectoHelpDesk
C:\proyectoHelpDesk>git --version
git version 2.54.0.windows.1
C:\proyectoHelpDesk>git config --global user.name
Pancho Reyes
C:\proyectoHelpDesk>git config --global user.email
freyes7194@utm.edu.ec
C:\proyectoHelpDesk>git config --global user.email
freyes7194@utm.edu.ec
C:\proyectoHelpDesk>git init
Initialized empty Git repository in C:/proyectoHelpDesk/.git/
C:\proyectoHelpDesk>git branch -M master
C:\proyectoHelpDesk>git bbranch
git: 'bbranch' is not a git command. See 'git --help'.
The most similar command is
branch
```



```
Git CMD
[master (root-commit) a9439a6] Iniciando mi repositorio
3 files changed, 42 insertions(+)
create mode 100644 diagramas/prueba de archjivo.txt
create mode 100644 pdf_actividad/prueba de archjivo.txt
create mode 100644 prueba de archjivo.txt

C:\proyectoHelpDesk>git branch
* master

C:\proyectoHelpDesk>git remote add origin https://github.com/freyes7194-svg/HelpDesk-Tarea.git

C:\proyectoHelpDesk>git push -u origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 576 bytes | 288.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/freyes7194-svg/HelpDesk-Tarea/pull/new/master
remote:
To https://github.com/freyes7194-svg/HelpDesk-Tarea.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

C:\proyectoHelpDesk>git status
On branch master
Your branch is up to date with 'origin/master'.
```

➤ Captura de las ramas master, develop y feature/.

```
Git CMD

C:\proyectoHelpDesk>git branch
feature/analizar_requerimiento
* featute/diagrama
master

C:\proyectoHelpDesk>git status
On branch featute/diagrama
nothing to commit, working tree clean

C:\proyectoHelpDesk>git checkout -b feature/diagrama
Switched to a new branch 'feature/diagrama'

C:\proyectoHelpDesk>git status
On branch feature/diagrama
nothing to commit, working tree clean

C:\proyectoHelpDesk>git add .

C:\proyectoHelpDesk>git push -u origin feature/diagrama
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/diagrama' on GitHub by visiting:
remote:   https://github.com/freyes7194-svg/HelpDesk-Tarea/pull/new/feature/diagrama
remote:
To https://github.com/freyes7194-svg/HelpDesk-Tarea.git
 * [new branch]      feature/diagrama -> feature/diagrama
branch 'feature/diagrama' set up to track 'origin/feature/diagrama'.

C:\proyectoHelpDesk>
```



Universidad

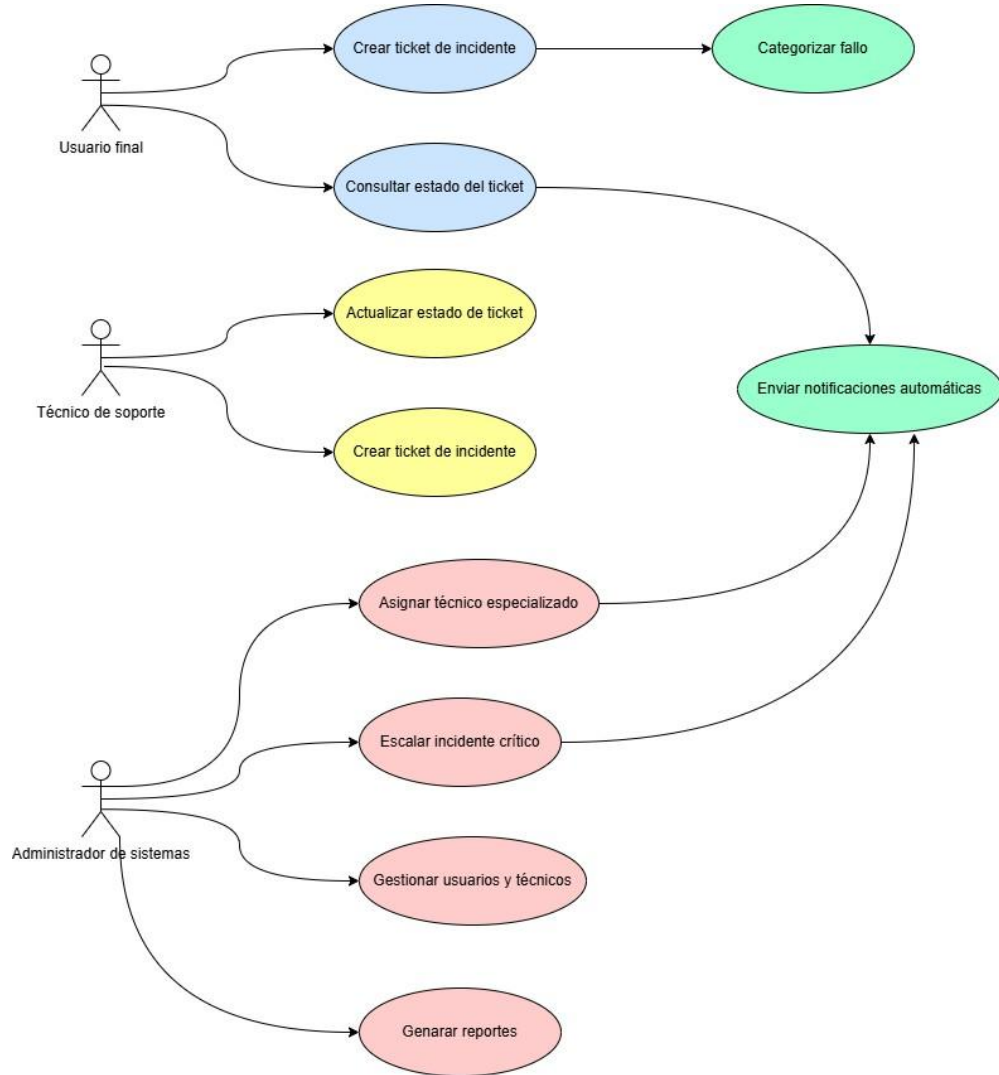
Técnica de Manabí

➤ Captura del historial de commits.

➤ Enlace al repositorio:

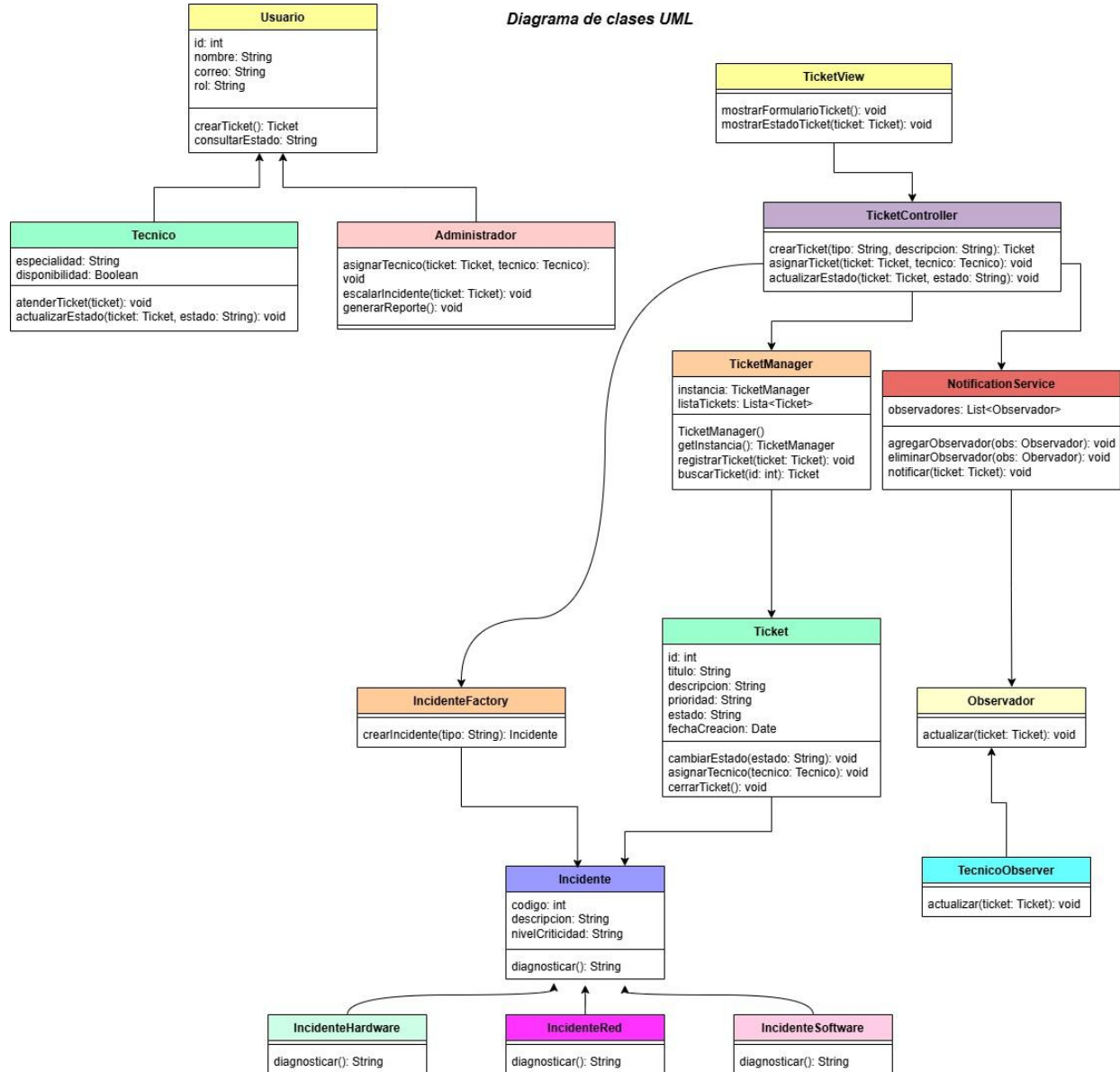
<https://github.com/freyes7194-svg/HelpDesk-Tarea.git>

➤ Diagrama de casos de uso

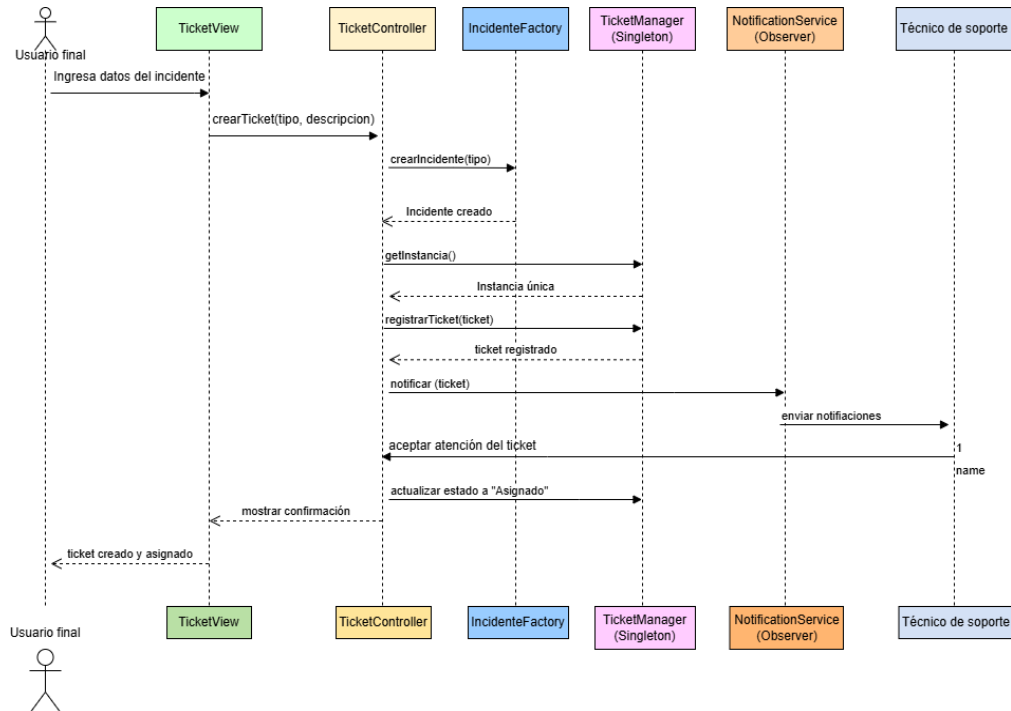


➤ Diagrama de clases UML

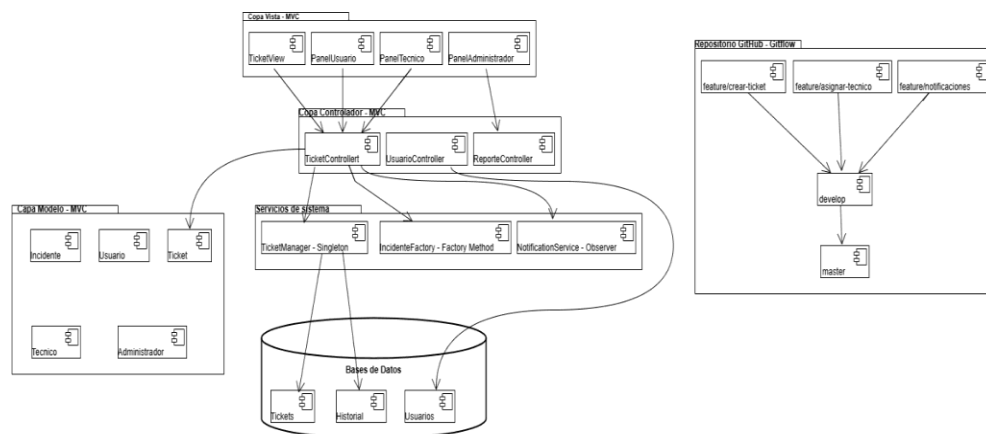
Diagrama de secuencia



➤ Diagrama de secuencia



➤ Diagrama de arquitectura general



Aplicación de patrones de diseño

Patrón Singleton



Propósito

El patrón Singleton permite garantizar que una clase tenga una única instancia global durante la ejecución del sistema. En este caso, se aplica al componente TicketManager, encargado de administrar los tickets registrados.

Aplicación en el sistema Help Desk

Para poder establecer un departamento funcional de atención al cliente.

marketing de atención al cliente. Si se diera el caso de que existiesen varios tickets managers funcionando al mismo tiempo, esto podría provocar redundancia o pérdida de consistencia en los estados de ticket.

Por eso, TicketManager se define como una única instancia compartida dentro del sistema.

Problema que resuelve

. Evitar que los controladores de entrada tengan múltiples gestores con diferente información, favorece el control centralizado del incidente.

Patrón Factory Method

➤ Propósito



Factory Method permite crear objetos sin depender directamente de clases concretas. En este proyecto se utiliza para crear distintos tipos de incidentes según su categoría: hardware, red o software.

➤ **Aplicación en el sistema Help Desk**

Cuando un usuario denuncia un incidente se debe identificar el sistema equivocado. El tener que crear manualmente cada objeto desde el controlador requiere mucho trabajo. Por tanto se delega a una fábrica que se llama IncidenteFactory.

➤ **Problema que resuelve**

Permite la adición de nuevos tipos de incidentes con ajustes mínimos en las funciones del sistema central. Las adiciones posteriores pueden incluir incidentes de seguridad, energía eléctrica o almacenamiento.

Patrón Observer

➤ **Propósito**

Observer proporciona una manera para que un objeto notifique a otros objetos cuando ocurre un cambio importante. Esto es un sistema de notificación.

➤ **Aplicación en el sistema Help Desk**

Cada vez que se genere un nuevo ticket o cambie de estado, se le debe notificar automáticamente el nuevo estado al técnico asignado o administrador.



Los técnicos actúan como observadores, mientras que el NotificationService es el sujeto que actúa como observable.

➤ **Problema que resuelve**

Impide que los técnicos tengan que revisar manualmente si hay un nuevo ticket.

Cuando ocurra un evento importante, el sistema se activará automáticamente.

Patrón MVC

➤ **Propósito**

Con MVC podemos separar la aplicación en tres partes que son modelo, vista y controlador. Esto permite tener más organizado el sistema, así como facilitar su mantenimiento.

➤ **Aplicación en el sistema Help Desk**

Componente MVC	Aplicación en el Help Desk
Modelo	Ticket, Incidente, Usuario, Técnico y Administrador.
Vista	Panel de creación de tickets de usuarios, panel de creación de tickets de técnicos y panel de creación de tickets de administrador.
Controlador	TicketController, UsuarioController y ReporteController.

➤ **Problema que resuelve**

Separa la interfaz de la lógica de negocio para que la vista y el modelo y/o las reglas de negocio puedan cambiar independientemente.



Relación con principios SOLID

Principio	Aplicación en el diseño
S - Responsabilidad única	Cada clase tiene una función específica. Por ejemplo, <code>Ticket</code> representa el ticket y <code>NotificationService</code> gestiona las notificaciones.
O - Abierto/Cerrado	Gracias a <code>IncidentFactory</code> se pueden agregar nuevos tipos de <code>_incidente</code> sin modificar todo el sistema.
L - Sustitución de Liskov	Las clases <code>IncidenteHardware</code> , <code>IncidenteRed</code> e <code>IncidenteSoftware</code> pueden sustituir a la clase abstracta <code>Incidente</code> .
I - Segregación de interfaces	La interfaz <code>Observer</code> tiene sólo un método para recibir las notificaciones.
D - Inversión de dependencias	El sistema no opera solo con clases concretas sino que lo hace con abstratos como <code>Incidente</code> y <code>Observador</code> .

Flujo de funcionamiento del sistema

- El usuario final ingresa al sistema Help Desk.
- Dé detalles de la falla técnica al crear un nuevo ticket.
- El incidente se clasifica como hardware, red o software por el sistema.
- Mediante Factory Method se crea el tipo de incidente correspondiente.
- El gestor de tickets central, cuya implementación está en Singleton, registra el ticket.
- El administrador o sistema asigna a un técnico según la especialidad que precisa la avería.
- El sistema manda una notificación automáticamente al técnico, a través de Observer.
- El técnico modifica el estado del ticket al atender el incidente.



- El usuario puede consultar el avance del caso.
- Una vez atendido el caso se marcará el ticket como culminado o cerrado.
- El sistema conserva un registro de todos los cambios realizados.

Estados del ticket

Estado	Descripción
Abierto	El usuario ha registrado el incidente pero aún no se ha asignado.
Asignado	El ticket ya cuenta con técnico asignado.
En proceso	El mecánico se encuentra corrigiendo el problema.
Escalado	Incidente que necesita atención de mayor nivel.
Resuelto	El profesionalista dejó escrita la solución.
Cerrado	El incidente fue validado y finalizado.

Ejemplo de flujo real de incidente

Un usuario reporta que un servidor del Data Center no responde. El sistema registra el ticket con prioridad alta y lo clasifica como incidente de red. Mediante el patrón Factory Method se crea un objeto `IncidenteRed`. Luego, el `TicketManager` registra el ticket en la lista central del sistema.

El administrador registra un ticket con problema de red y lo pasa a un técnico para su gestión. El técnico adjunto es notificado automáticamente, gracias al módulo de notificaciones basado en Observer. Después de que el técnico revisa el caso, identifica que hay una falla en la conectividad del switch, así que cambia el estado del ticket a “En proceso”. Tras corregir la avería, el técnico registra la acción correctiva, lo que cambia el estado del caso a "Resuelto". Por



último se cierra la validación del usuario se cierra.

README sugerido para GitHub

Sistema Help Desk para Data Center

Proyecto académico orientado al diseño lógico de un sistema de gestión de incidentes técnicos para infraestructura de servidores en un Data Center.

Objetivo

Diseño de arquitectura lógica para un registro, clasificación, asignación y seguimiento de incidentes técnicos, aplicando el patrón de diseño de software y el flujo de trabajo Gitflow.

Actores principales

Usuario final

Técnico de soporte

Administrador de sistemas

Patrones de diseño aplicados

Singleton: Gestión de tickets centralizada. Reporte de daños en la fábrica y/o operadores.

Observer: Notificaciones automáticas.

MVC: división del modelo, vista y controlador



Ramas Gitflow

master develop

feature/analizar_requerimiento feature/desarrollo feature/implementación

feature/diagrama

Conclusiones

- El diseño del sistema Help Desk propuesto permite gestionar de manera organizada los incidentes técnicos dentro de un Data Center. La identificación de actores, requerimientos y flujos de trabajo ayuda a comprender las necesidades reales del sistema y su relación con la infraestructura tecnológica.
- El uso de patrones de diseño mejora la arquitectura del proyecto. Singleton facilita la concentración de gestión de tickets, el patrón Factory Method permite crear un producto sin tener que especificar la clase que se va a utilizar. El patrón Observer otorga autonomía.
- El uso de Gitflow aporta trazabilidad, orden y control en el proceso de desarrollo, ya que permite trabajar con ramas separadas para funcionalidades, integración y versiones estables. Esto representa una práctica profesional importante en el desarrollo de sistemas informáticos.
- En conjunto se cumplen lo esperado de los sistemas con escalabilidad, mantenibilidad, reusabilidad, claridad arquitectónica, una solución importante para el software profesional.



Bibliografía

Draw.io – Herramienta para la elaboración de diagramas UML y arquitecturas.

<https://www.drawio.com/>

Git – Sistema de control de versiones distribuido. <https://git-scm.com/>

GitHub – Plataforma para repositorios remotos y gestión colaborativa de proyectos.

<https://github.com/>



Universidad

Técnica de Manabí



Universidad

Técnica de Manabí



Universidad

Técnica de Manabí



Universidad
Técnica de Manabí
